

Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title:** [Your Project Title]
- **Team Members:** List team members and their roles.

2. Project Overview

- **Purpose:** Briefly describe the purpose and goals of the project.
- **Features:** Highlight key features and functionalities.

3. Architecture

- **Frontend:** Describe the frontend architecture using React.
- **Backend:** Outline the backend architecture using Node.js and Express.js.
- **Database:** Detail the database schema and interactions with MongoDB.

4. Setup Instructions

- **Prerequisites:** List software dependencies (e.g., Node.js, MongoDB).
- **Installation:** Step-by-step guide to clone, install dependencies, and set up the environment variables.

5. Folder Structure

- **Client:** Describe the structure of the React frontend.
- **Server:** Explain the organization of the Node.js backend.

6. Running the Application

- Provide commands to start the frontend and backend servers locally.
- o **Frontend:** npm start in the client directory.
- o **Backend:** npm start in the server directory.

7. API Documentation

- Document all endpoints exposed by the backend.
- Include request methods, parameters, and example responses.

8. Authentication

- Explain how authentication and authorization are handled in the project
- Include details about tokens, sessions, or any other methods used.

9. User Interface

- Provide screenshots or GIFs showcasing different UI features.

10. Testing

- Describe the testing strategy and tools used.

11. Screenshots or Demo

- Provide screenshots or a link to a demo to showcase the application.

12. Known Issues

- Document any known bugs or issues that users or developers should be aware of.

13. Future Enhancements

- Outline potential future features or improvements that could be made to the project.

OrderOnTheGo – On-Demand Food Ordering Platform.

Introduction

Project Title: OrderOnTheGo – On-Demand Food Ordering Platform

Team Details:

Team ID: LTVIP2026TMIDS81358

Team Leader: Poojitha Poshimshetti

2. Project Overview:

Purpose of the Project

Traditional food ordering methods often involve manual phone calls or unstructured ordering processes, which can lead to delays and miscommunication. OrderOnTheGo aims to provide a modern digital solution that streamlines restaurant management, enhances customer convenience, and improves order tracking.

The application is built using MongoDB, Express.js, React.js, and Node.js, following RESTful API architecture and secure authentication practices.

Goals of the Project

1. Develop a centralized online food ordering system.
2. Implement secure JWT-based authentication.
3. Provide role-based dashboards for Admin, Restaurant, and Customer.
4. Ensure scalable architecture using MongoDB Atlas cloud database.
5. Improve user experience with a responsive and intuitive interface.

Key Features and Functionalities:

Customer Features

- Secure registration and login using JWT authentication.
- Browse restaurant menus and food items.
- Add items to cart and place orders.
- View order history and status updates.

Restaurant Features

- Manage food menu and product listings.
- View incoming orders.
- Update order status (Preparing, Delivered).

Admin Features

- Approve restaurant registrations.
- Monitor users and platform activity.
- Manage roles and system governance.

Technical Features

- RESTful API Architecture.
- MongoDB Atlas Cloud Database.
- Role-Based Access Control.
- MVC Backend Structure.

3. Architecture

I. Web Application (Frontend Layer):

The frontend is developed using React.js with a component-based architecture.

Key frontend responsibilities include:

- Rendering dashboards for different roles.
- Managing routing using React Router.
- Handling API communication using Axios.
- Provide responsive UI using Bootstrap and modern UI libraries.

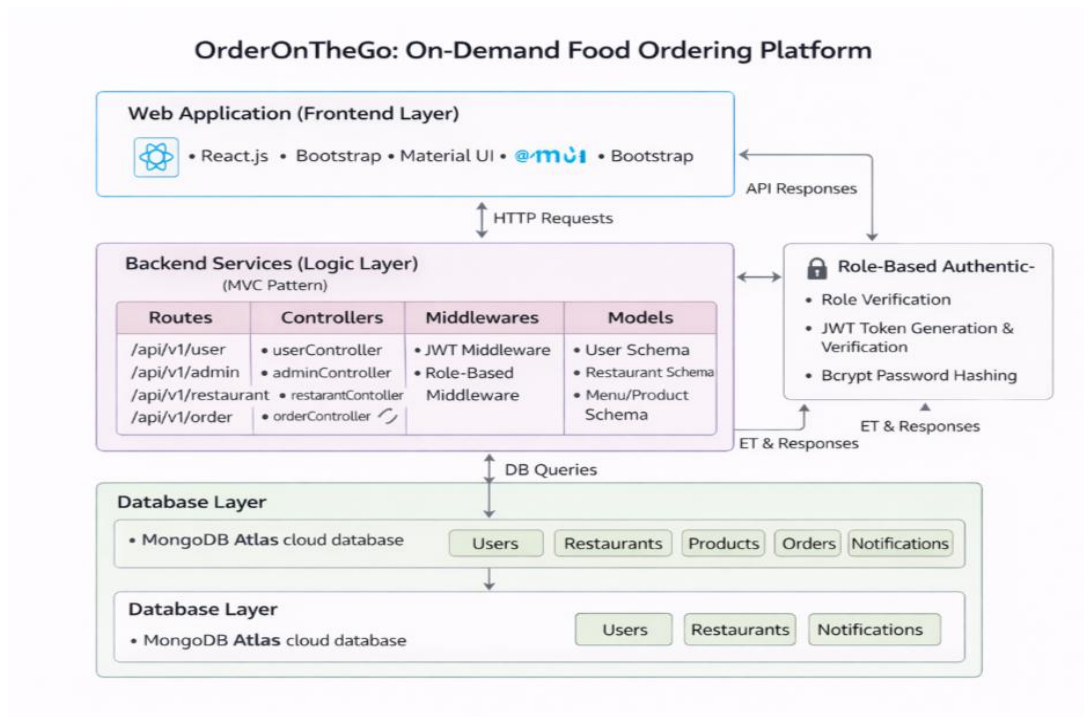


Fig-1: Main Architecture

II. Backend Services (Logic Layer)

The backend is built using Node.js and Express.js following an MVC architecture.

Components:

- **Routes:** Define API endpoints.
- **Controllers:** Business logic for authentication, orders, and admin actions.
- **Middleware:** JWT authentication and role validation.
- **Schemas:** Mongoose models for MongoDB collections.

This layered structure ensures maintainability, scalability, and secure request processing.

III. Database Layer (MongoDB Atlas)

The application uses MongoDB Atlas as a cloud database.

Collections include:

- Users
- Restaurants
- Products
- Orders
- Notification

4. Setup Instructions

Prerequisites:

To develop the OrderOnTheGo full-stack food ordering web application using React.js, Node.js, Express.js, and MongoDB, the following prerequisites are required:

Node.js and npm: Node.js is required to run JavaScript on the server side and manage project dependencies using npm.

- Download: <https://nodejs.org/en/download/>
- Installation instructions: <https://nodejs.org/en/download/package-manager/>

MongoDB: MongoDB Atlas is used as the cloud database to store users, restaurants, products, and order information.

- Download: <https://www.mongodb.com/try/download/community>
- Installation instructions: <https://docs.mongodb.com/manual/installation/>

Express.js: Express.js is a backend framework used for API routing, middleware handling, and server-side logic.

- Install Express inside the backend project folder using: `npm install express`

React js: React is used to build the frontend user interface of OrderOnTheGo, enabling a fast and responsive single-page application.

Installation Overview

To run the OrderOnTheGo application locally:

1. Clone the project repository.
2. Install frontend and backend dependencies.
3. Configure environment variables including database URL and JWT secret key.
4. Run both frontend and backend servers simultaneously.

Getting Started

Create React App is an officially supported way to create single-page React applications. It offers a modern build setup with no configuration.

Quick Start

```
npm create vite@latest
```

```
cd orderonthego-client
```

```
npm install
```

```
npm run dev
```

If you've previously installed create-react-app globally via `npm install -g create-react-app`, we recommend you uninstall the package using `npm uninstall -g create-react-app` or `yarn global remove create-react-app` to ensure that npx always uses the latest version.

Create a new React project:

To build the frontend of the OrderOnTheGo food ordering platform, a React.js application is used to develop a fast and responsive user interface.

- Choose or create a directory where you want to set up the OrderOnTheGo frontend project.
- Open your terminal or command prompt.
- Navigate to the selected directory using the `cd` command.
- Create a new React project by running the following command: `npx create-react-app orderonthego-client`. Wait for the project to be created:
- This command will generate the basic project structure and install the necessary dependencies

Navigate into the project directory:

- After the project creation is complete, navigate into the project directory by running the following command: `cd orderonthego-client`

Start the development server:

- To launch the development server and see your React app in the browser, run the following command: `npm run dev`
- The npm start will compile your app and start the development server.
- Open your web browser and navigate to <http://localhost:3000> to see your React app.

You have successfully set up React on your machine and created a new React project. You can now start building your app by modifying the generated project files in the `src` directory.

Please note that these instructions provide a basic setup for React. You can explore more advanced configurations and features by referring to the official React documentation: <https://react.dev/>

HTML, CSS, and JavaScript: Basic understanding of HTML, CSS, and JavaScript is essential for: Designing responsive layouts, Styling food menus and order screens, Handling client-side interactions.

Database Connectivity: The backend server connects to MongoDB Atlas using Mongoose ODM. This enables CRUD operations such as: Managing restaurants and menus, Processing food orders, Handling user authentication data.

Front-end Library: React.js is used to build the OrderOnTheGo user interface, including: Menu listings, Place Order” functionality, Restaurant dashboards, Admin management panels.

Version Control: Use Git for version control, enabling collaboration and tracking changes throughout the development process. Platforms like GitHub or Bitbucket can host your repository.

Git: Download and installation instructions can be found at: <https://git-scm.com/downloads>

Development Environment: Visual Studio Code is recommended as the primary IDE for developing the OrderOnTheGo platform.

- Visual Studio Code: Download from <https://code.visualstudio.com/download>

5. Folder Structure

The OrderOnTheGo project follows a modular full-stack architecture and is organized into two main modules: Client (Frontend) and Backend (Server).

Client Module:

The client module is responsible for rendering the user interface and handling all user interactions within the platform. It is developed using React.js with a component-based architecture.

Customer Dashboard – Browse restaurants, view menus, and place orders

Restaurant Panel – Manage orders, update menus, and track order status

Admin Panel – Monitor users, restaurants, and platform activities

Backend Module

The backend module handles business logic, database communication, authentication, and API routing. It follows an MVC-based structure for better organization.

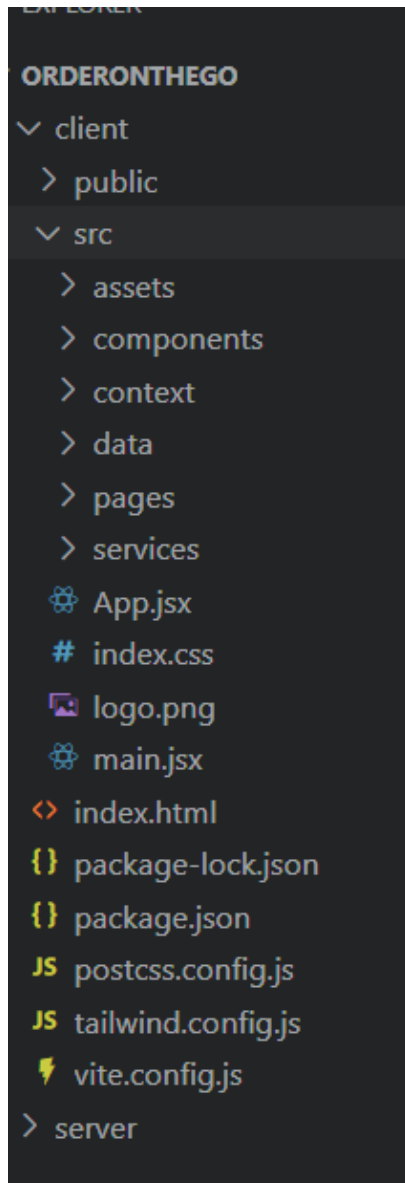


Fig-2: Client Structure

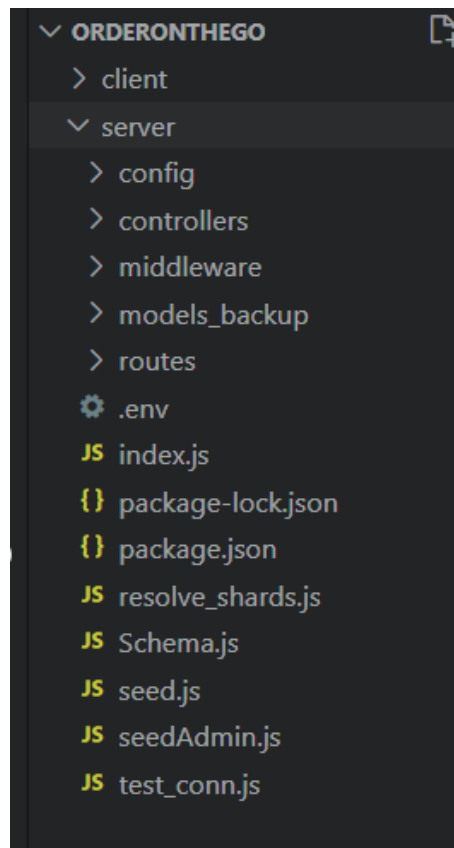


Fig-3: Server Structure

Frontend (React Client) – Structure

The frontend of the OrderOnTheGo application is developed using React.js and follows a modular, component-based architecture that promotes scalability, reusability, and maintainability. The client layer serves as the main interface where customers, restaurants, and administrators interact with the platform.

The React client is organized into structured folders such as pages, components, redux store, routes, and services. The pages directory contains role-based interfaces including Customer Dashboard, Restaurant Panel, Admin Panel, authentication pages, cart management, and order tracking screens. Each page represents a functional module responsible for interacting with backend APIs.

Reusable UI components such as navigation bars, loaders, protected routes, modals, and layout wrappers are stored within the components folder. These shared components ensure consistent user experience across different modules of the application.

Global state management is handled using Redux Toolkit, allowing centralized data flow for cart items, user sessions, restaurant menus, and order status updates. React Hooks such as `useState`, `useEffect`, `useSelector`, and `useDispatch` are used to manage UI state, lifecycle behavior, and API responses efficiently.

Backend (Node.js Server) – Structure:

The backend of OrderOnTheGo is built using Node.js and Express.js, following a structured MVC (Model-View-Controller) architecture. This approach separates routing logic, business operations, and database models into independent layers, improving maintainability and scalability.

The backend directory is organized into folders such as config, controllers, middlewares, routes, models, and uploads.

The models folder defines MongoDB data schemas using Mongoose. It includes schema definitions for Users, Restaurants, Menu Items, Orders, and Notifications, which represent the core entities of the food ordering system.

The routes folder manages REST API endpoint definitions. Each route file corresponds to a specific module such as `userRoutes`, `restaurantRoutes`, `adminRoutes`, and `orderRoutes`. These routes receive HTTP requests and delegate processing to appropriate controllers.

Controllers contain the main business logic of the application. They handle:

- User authentication and account management
- Restaurant menu management
- Order placement and status updates
- Admin approval workflows
- Notification handling

The middlewares folder includes authentication and authorization logic. JWT middleware verifies user identity, while role-based middleware restricts access to APIs based on roles such as Customer, Restaurant, and Admin. Passwords are securely stored using bcrypt hashing.

The config folder manages database connection settings and environment configurations, enabling secure integration with MongoDB Atlas cloud database.

RESTful API principles are followed throughout the backend, allowing standardized communication between frontend and server. Express.js handles routing, middleware execution, and response handling to ensure efficient performance.

This backend architecture enhances security, supports scalable food ordering workflows, and allows future expansion such as real-time order tracking or payment integration.

Database (MongoDB Atlas) – Structure:

The OrderOnTheGo platform uses MongoDB Atlas as a cloud-based NoSQL database system. MongoDB stores data in flexible JSON-like documents, enabling dynamic schema updates and scalable performance.

The database consists of multiple collections:

- Users Collection Stores customer and admin account details, authentication credentials, and order history.
- Restaurants Collection Maintains restaurant profile information, menus, availability status, and approval details.
- Orders Collection Records order details, selected menu items, delivery status, and payment information.
- Notifications Collection Manages system alerts, order updates, and communication between users, restaurants, and admins.

6. Running the Application

To run the OrderOnTheGo Food Ordering Platform successfully without errors, both the backend server and frontend client must be executed simultaneously. The backend handles API requests, authentication, restaurant management, and order processing, while the frontend provides the user interface for customers, restaurants, and administrators. To run these both, we need the commands, the commands are :

For Backend : First navigate into the folder using the command

“cd folder name(Server) “

Next, run the command

“npm start “.

For Frontend: First navigate into the folder using the command

“cd folder name(Client) “

Next, run the command

“npm run dev “

7. API Documentation

The OrderOnTheGo backend exposes multiple RESTful API endpoints that manage authentication, restaurant operations, food ordering, notifications, and administrative control. All APIs follow standard HTTP methods such as GET and POST, and responses are returned in JSON format.

Protected routes use JWT authentication middleware to verify user identity and enforce role-based authorization for Customer, Restaurant, and Admin roles.

1. Authentication & User APIs

1.1 Register User

Endpoint:

POST /api/v1/user/register

Description:

Creates a new customer account in the system.

Request Body:

```
{  
  "name": "John Doe",  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

Response:

```
{  
  "message": "User registered successfully"  
}
```

1.2 Login User

Endpoint:

POST /api/v1/user/login

Description:

Authenticates user credentials and generates a JWT token.

Request Body:

```
{  
  "email": "john@gmail.com",  
  "password": "123456"  
}
```

Response:

```
{  
  "success": true,  
  "token": "jwt_token_here",  
  "user": {  
    "_id": "12345",  
    "email": "john@gmail.com"  
  }  
}
```

1.3 Get User Data (Protected)**Endpoint:**

POST /api/v1/user/getUserData

Description:

Returns authenticated user profile details.

Authorization Required: JWT Token

1.4 Apply as Restaurant**Endpoint:**

POST /api/v1/user/apply-restaurant

Description:

Allows a user to submit a restaurant registration request for admin approval.

1.5 Notifications APIs**Get All Notifications**

POST /api/v1/user/get-all-notification

Delete All Notifications

POST /api/v1/user/delete-all-notification

Description:

Fetches or clears notification history such as order updates or admin alerts.

2. Restaurant APIs

2.1 Get Restaurant Info

Endpoint:

POST /api/v1/restaurant/getRestaurantInfo

Description:

Fetches profile details of the logged-in restaurant owner.

2.2 Update Restaurant Profile

Endpoint:

POST /api/v1/restaurant/updateProfile

Description:

Updates restaurant details such as name, address, cuisine type, and operating hours.

2.3 Get Restaurant By ID

Endpoint:

POST /api/v1/restaurant/getRestaurantById

Description:

Returns restaurant information using its unique ID.

3. Order APIs

3.1 Place Order

Endpoint:

POST /api/v1/order/place-order

Description:

Creates a new food order for a customer.

Request Body:

```
{  
  "restaurantId": "rest123",  
  "items": ["pizza", "burger"],  
  "totalAmount": 500  
}
```

Response:

```
{  
  "message": "Order placed successfully"  
}
```

3.2 Check Restaurant Availability**Endpoint:**

POST /api/v1/order/check-availability

Description:

Checks whether a restaurant is currently accepting orders.

3.3 Get User Orders**Endpoint:**

GET /api/v1/order/user-orders

Description:

Returns order history for the logged-in customer.

3.4 Get Restaurant Orders**Endpoint:**

GET /api/v1/order/restaurant-orders

Description:

Returns order list for restaurant dashboard management

3.5 Update Order Status**Endpoint:**

POST /api/v1/order/update-status

Description:

Allows restaurant owners to accept, prepare, or deliver food orders.

4. Admin APIs

4.1 Get All Users**Endpoint:**

GET /api/v1/admin/getAllUsers

Description:

Fetches all registered customers for system monitoring.

4.2 Get All Restaurants**Endpoint:**

GET /api/v1/admin/getAllRestaurants

Description:

Returns restaurant list including approval status.

4.3 Change Restaurant Account Status**Endpoint:**

POST /api/v1/admin/changeAccountStatus

Description:

Admin approves or rejects restaurant registration requests.

4.4 Change User Status**Endpoint:**

POST /api/v1/admin/change-user-status

Description:

Updates customer account status within the system.

Authentication Requirement

All protected endpoints require a valid JWT token in request headers:

Authorization: Bearer <token>

JWT middleware verifies identity and enforces **role-based access** for Customer, Restaurant, and Admin dashboards, ensuring secure communication between frontend and backend services.

8. Authentication

Authentication and Authorization in OrderOnTheGo

The OrderOnTheGo application implements a secure authentication and authorization system using JSON Web Tokens (JWT) along with Role-Based Access Control (RBAC). The system ensures that only authenticated users can access protected resources such as order placement, restaurant dashboards, and administrative controls.

Authentication verifies the identity of users, while authorization determines the level of access based on roles such as Admin, Restaurant Owner, and Customer. This layered security approach protects user data, order information, and platform operations from unauthorized access.

Authentication Process

Authentication in OrderOnTheGo follows a token-based architecture. When a user logs in with valid credentials, the backend server verifies the user's email and password stored in the MongoDB Atlas database. Passwords are encrypted using bcrypt hashing to ensure secure storage.

After successful verification, the server generates a JSON Web Token (JWT) containing essential user information such as the user ID and role. This token is digitally signed using a secret key and returned to the frontend application.

The frontend stores the token securely and attaches it to future API requests that require authentication.

Example header format:

Authorization: Bearer <JWT_Token>

Before processing any protected request, backend authentication middleware validates the token to ensure it is legitimate and has not expired.

JWT (JSON Web Token) Usage

The JWT used in OrderOnTheGo contains encoded payload information including:

- User ID
- User Role (admin, restaurant, customer)
- Token expiration timestamp

Using JWT enables:

- Stateless authentication without maintaining server sessions.
- Secure request verification without repeated database queries.
- Efficient communication between frontend and backend services.

Each protected API endpoint uses authentication middleware that performs the following steps:

1. Extracts the token from request headers.
2. Verifies the token using a secret key.
3. Decodes the payload data.
4. Attaches authenticated user information to the request object.

If the token is invalid, missing, or expired, the server denies access and returns an unauthorized response.

Authorization (Role-Based Access Control)

OrderOnTheGo implements Role-Based Access Control (RBAC) to restrict access to sensitive operations depending on user roles.

Every user account contains a role field that determines permissions:

- **Customer:** Can browse restaurants, add items to cart, place orders, and view order history.
- **Restaurant Owner:** Can manage menu items, update order status, and handle incoming orders.
- **Admin:** Can approve restaurant registrations, manage users, and oversee system operations.

Authorization middleware verifies the role of the authenticated user before allowing access to specific routes.

Examples include:

- Only Admin users can approve restaurant accounts.
- Only Restaurant Owners can update order status.
- Only authenticated Customers can place orders.

If a user attempts to access restricted routes without proper permissions, the system returns an authorization error.

Session Handling

The OrderOnTheGo platform uses stateless authentication through JWT tokens instead of traditional server-side sessions. This improves scalability and performance by eliminating the need to store session data on the server.

Since each request carries authentication details within the token, the backend validates requests independently without maintaining session state.

Security Measures Implemented

Several security practices are applied to protect the OrderOnTheGo system:

- Passwords are hashed using bcrypt before storing in MongoDB Atlas.
- JWT tokens are signed with a secure secret key.
- Protected routes validate tokens through authentication middleware.
- Role-based authorization prevents unauthorized access to sensitive endpoints.
- Token expiration mechanisms automatically invalidate inactive sessions.

These measures ensure data confidentiality, integrity, and controlled access throughout the application lifecycle.

9. User Interface:

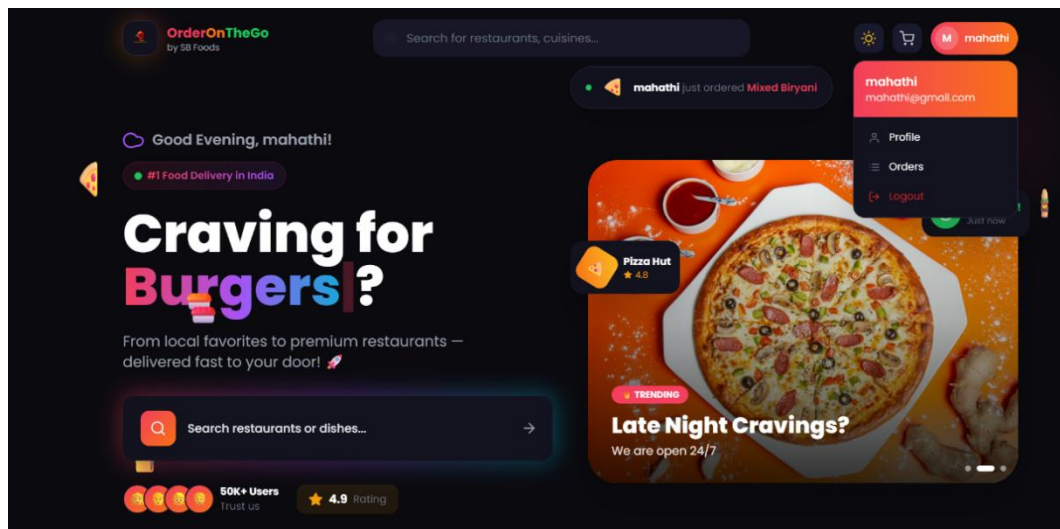


Fig-4: Customer Dashboard

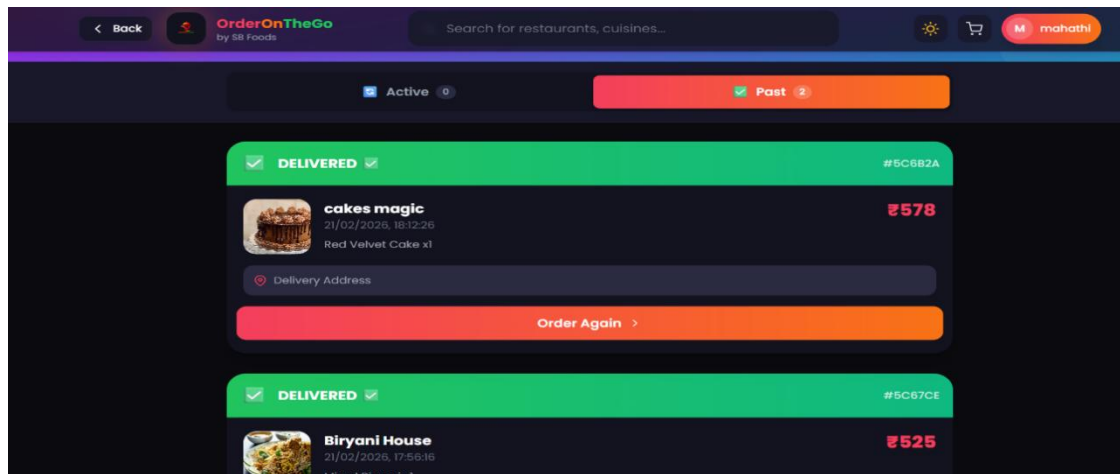


Fig-4: Order History

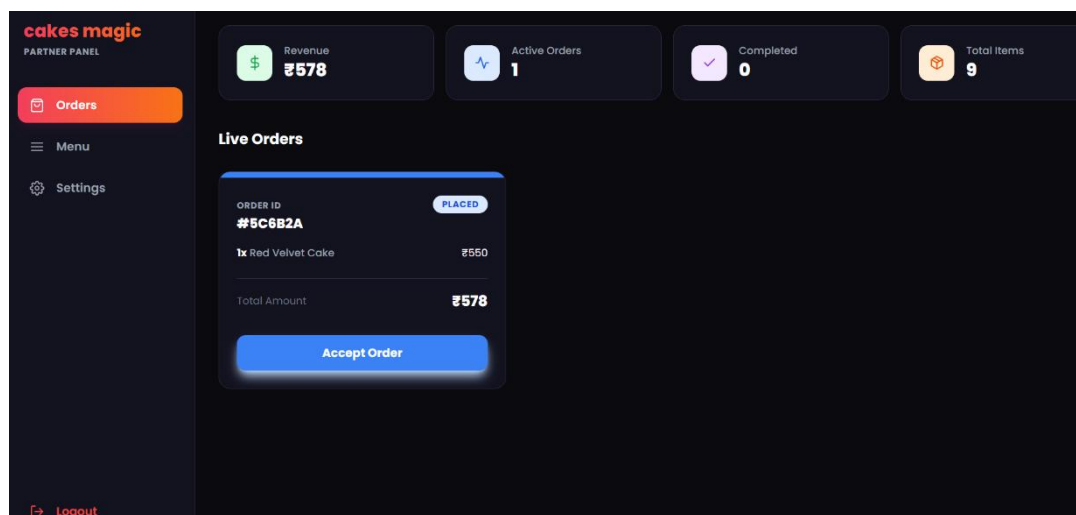


Fig-5: Restaurant Dashboard

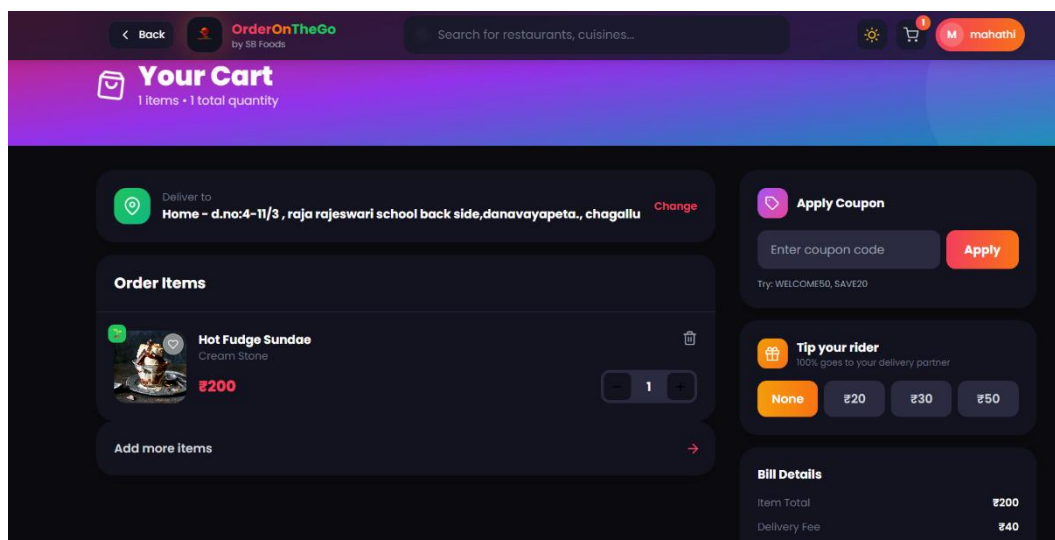


Figure-5: Cart Page

10. Testing

Testing Strategy and Tools Used

Testing in the OrderOnTheGo project was conducted to ensure that the food ordering application functions correctly, securely, and efficiently across both frontend and backend components. The testing strategy included manual testing, functional testing, API testing, authentication testing, UI testing, and error-handling validation.

1. Manual Testing

Manual testing was performed during development to verify the functionality of each module. Features such as authentication, restaurant management, cart operations, and order processing were tested individually.

Examples:

- Testing login with valid and invalid credentials.
- Checking restaurant menu loading and food item filtering.
- Verifying order status updates by restaurant owners.
- Ensuring cart updates correctly when items are added or removed.

2. Functional Testing:

Functional testing ensured that all application features behaved according to system requirements. Each workflow was tested from the perspective of Customers, Restaurants, and Admin users.

Modules tested include:

- User Registration and Login
- Restaurant Menu CRUD operations
- Cart management
- Checkout and order placement
- Restaurant Dashboard Controls
- Admin Panel Management

3. API Testing

Backend APIs were tested using tools such as:

- Postman (for testing REST API endpoints)

- Browser developer tools (Network tab)

Each API endpoint was tested with:

- Valid request data
- Missing fields
- Unauthorized access attempts
- Invalid tokens

This ensured proper error handling and secure API behavior.

4. Authentication and Authorization Testing

JWT token verification was tested by:

- Accessing protected routes without a token.
- Using expired or invalid tokens.
- Attempting Admin or Restaurant operations with Customer accounts.

The system correctly restricted unauthorized access, confirming proper role-based control.

5. UI Testing

Frontend UI testing was performed to ensure:

- Proper rendering of components.
- Responsive layout behavior.
- Correct display of restaurant menus and food images.
- Smooth navigation between dashboards.
- Error message display for failed operations.

6. Error Handling Testing

The system was tested for:

- Server errors
- Database connection failures
- Invalid inputs

- Network interruptions

Proper error messages were displayed to the user without crashing the application.

Tools Used for Testing

- Postman – for API endpoint testing
- Browser Developer Tools – for debugging frontend
- MongoDB Compass – for verifying database records
- Console Logging – for debugging application flow

11. Screenshots or Demo

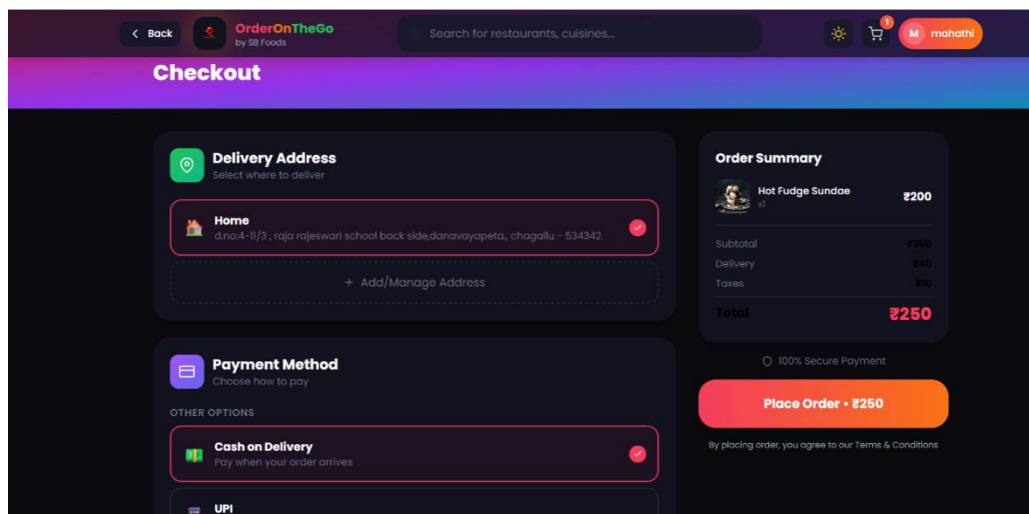


Figure-6: Checkout Page

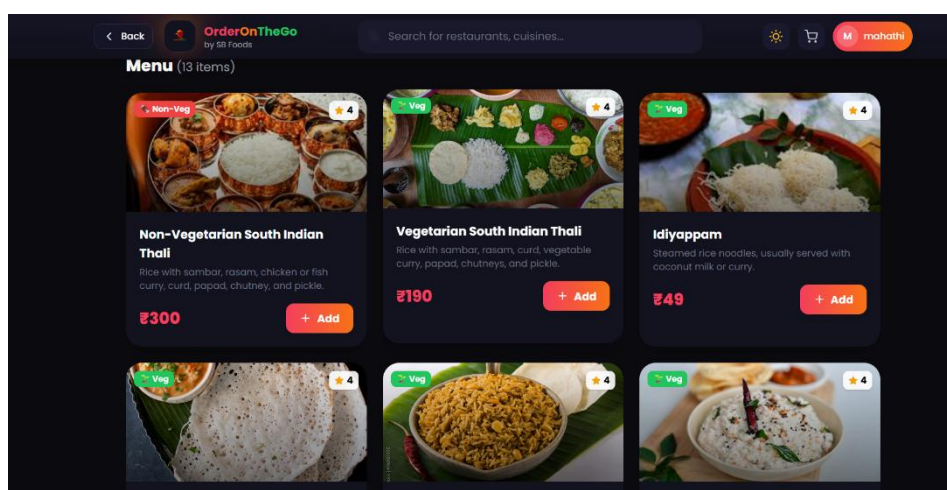


Figure-7: Restaurant Menu Listing Page

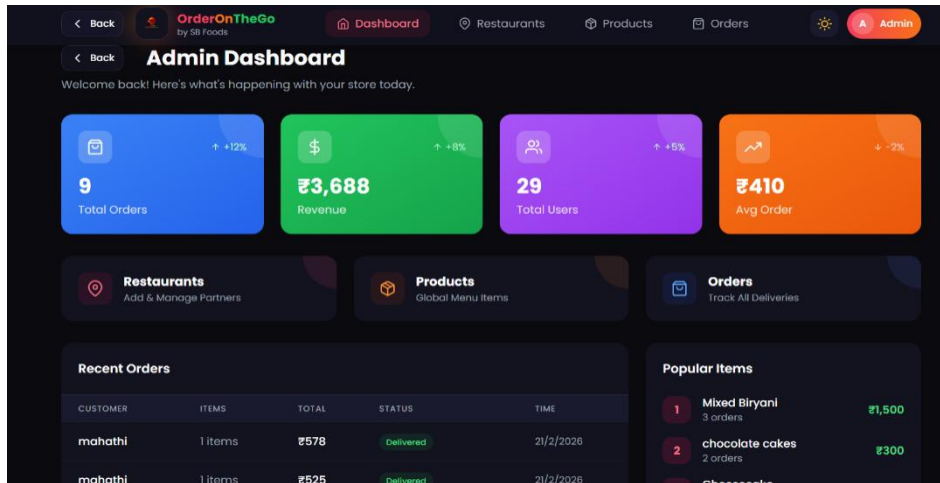


Figure-8: Admin Dashboard

12. Known Issues

Although the OrderOnTheGo application successfully implements the core functionalities of an online food ordering platform, certain limitations and known issues may affect usability, performance, and scalability.

1. No Real-Time Order Updates

Currently, order status updates are reflected only after refreshing the page. The system does not include real-time communication mechanisms such as WebSockets or push notifications.

Impact:

Customers may not instantly see updates when a restaurant changes the order status (e.g., Preparing or Delivered).

2. JWT Token Storage in Local Storage

Authentication tokens are stored in browser local storage for maintaining login sessions.

Impact:

While suitable for development and academic projects, storing tokens in local storage may expose potential security risks such as XSS attacks in production environments.

3. Limited Mobile Responsiveness

The user interface is optimized mainly for desktop usage. Some layouts may not fully adapt to smaller mobile screens.

Impact:

Mobile users may experience minor alignment or spacing issues while browsing menus or placing orders.

4. Food Image Loading Performance

Loading restaurant and menu images depends on network speed and image size.

Impact:

Large image files may increase loading time, affecting the browsing experience for users with slower internet connections.

5. No Online Payment Integration

The current version focuses on food ordering and order management but does not include integrated payment gateways.

Impact:

The application is suitable for demonstration and learning purposes but requires payment integration (UPI, Cards, Wallets) for real-world deployment.

6. Localhost Deployment Only

The application currently runs on a local development server and is not deployed on a production cloud environment.

Impact:

External users cannot access the system remotely without deployment configuration.

13. Future Enhancements

Although the OrderOnTheGo platform successfully implements the core functionality of an online food ordering system, several enhancements can be introduced in future versions to improve usability, security, scalability, and overall user experience.

1. Real-Time Order Tracking

Future versions of OrderOnTheGo can integrate real-time order tracking features that allow customers to view live updates on their food orders. Technologies such as WebSockets or Firebase Realtime Database can be used to provide instant status updates without page refresh.

2. Advanced UI/UX Improvements

The current interface focuses mainly on core functionality. Future updates may include improved responsive design, smoother animations, dark mode support, and accessibility features using modern UI frameworks to enhance the overall user experience.

3. Social Media Authentication

Integration of OAuth-based authentication such as Google or LinkedIn login can simplify the registration and login process while maintaining secure authentication workflows.

4. Real-Time Notifications System

Implementing real-time notifications using WebSockets or push notification services would allow customers and restaurant owners to receive instant updates about order confirmations, preparation status, and delivery progress.

5. Online Payment Integration

Future versions can include secure digital payment gateways such as UPI, debit/credit cards, or digital wallets. This enhancement will allow customers to complete transactions directly through the platform, making the system production-ready.

6. Enhanced Security Mechanisms

Instead of storing JWT tokens in local storage, future versions can implement HTTP-only cookies, CSRF protection, and advanced encryption practices to improve security against common vulnerabilities.

7. Performance Optimization

System performance can be enhanced through:

- Database indexing for faster product and order queries.
- Backend caching strategies to reduce API response time.
- Lazy loading of menu images and UI components.
- Pagination for large product listings and order histories.

8. Mobile Application Development

A dedicated mobile application using **React Native** or **Flutter** can extend OrderOnTheGo's accessibility to Android and iOS users, allowing customers to browse menus, place orders, and track delivery from mobile devices.

Project Video Demo

The Demo of the Application is available at:

<https://drive.google.com/file/d/1afy-lmsBbqmpWzGKKzhpBWvjxShjguyc/view?usp=drivesdk>